

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Реализация метода Гаусса с выбором ведущего элемента для решения СЛАУ.»

Выполнил:

студент группы 3825Б1ПМИИ1
Лебедев Сергей Алексеевич

Проверила:

Бусько П.В.

Содержание

Введение	3
Постановка задачи.....	4
Руководство пользователя	5
Описание программной реализации	6
Результаты экспериментов	9
Заключение.....	12
Приложение	13

Введение

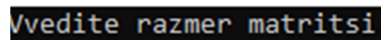
В вычислительной математике одним из наиболее эффективных методов решения систем линейных алгебраических уравнений является метод Гаусса. Этот метод основан на последовательном исключении неизвестных и позволяет привести исходную систему к треугольному виду, после чего решение находится с помощью обратного хода. Благодаря своей простоте и надежности метод Гаусса широко применяется в научных и инженерных расчетах. Однако при практической реализации метода на компьютере возникает ряд проблем, связанных с особенностями представления вещественных чисел. Как известно, числа с плавающей точкой имеют ограниченную точность, что приводит к неизбежному появлению погрешностей округления. Особый интерес представляет вопрос о выборе ведущего элемента – при неудачном выборе возможна потеря точности или даже деление на ноль. В связи с этим возникает необходимость реализации метода Гаусса с выбором ведущего элемента по столбцу. Такой подход позволяет минимизировать влияние погрешностей округления и обеспечить корректную работу алгоритма даже при наличии нулевых диагональных элементов в исходной матрице. Дополнительным преимуществом является возможность обнаружения вырожденных систем, для которых решение не существует или не единственно.

Постановка задачи

В данной лабораторной работе требуется разработать программную реализацию решения систем линейных алгебраических уравнений методом Гаусса. Необходимо реализовать шаблонный класс для работы с динамическими векторами, шаблонный класс для работы с квадратными матрицами, а также класс СЛАУ, наследующий класс матрицы. При разработке программы обязательно использование шаблонов для обеспечения работы с различными типами данных, а также механизма исключений для обработки вырожденных матриц. В ходе выполнения работы требуется реализовать метод Гаусса с выбором ведущего элемента по столбцу для повышения численной устойчивости алгоритма. Для этого необходимо выполнить перестановку строк при обнаружении максимального элемента в столбце и обеспечить корректное решение системы с последующим выводом результата.

Руководство пользователя

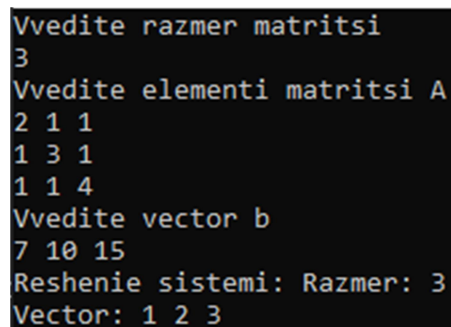
При запуске программы перед пользователем появляется запрос ввести размер элемента, показанный на рис. 1.



```
Vvedite razmer matritsi
```

Рис. 1 – Выбор размера матрицы

Далее следует ввести матрицу. Вводить ее нужно по строкам в соответствии с выбранным размером. Например, если введен размер 3 то, пользователь должен ввести три числа через пробел, после того как ввел, следует их отправить и продолжить вводить оставшиеся строки. Далее вводится вектор *b* аналогично матрице. На рис. 2 представлен пример работы программы.



```
Vvedite razmer matritsi
3
Vvedite elementi matritsi A
2 1 1
1 3 1
1 1 4
Vvedite vector b
7 10 15
Reshenie sistemi: Razmer: 3
Vector: 1 2 3
```

Рис. 2 – Пример работы программы

Описание программной реализации

В проекте есть 2 файла: header.h, main.cpp. В файле main.cpp описана функция main. В файле header.h перечислены все классы и функции, которые используются в программе, а также подключены библиотеки iostream, smath и stdexcept. Далее будут описаны все классы и функции из этого файла.

```
template <typename T> class Moevector
```

Шаблонный класс, предназначенный для работы с динамическим вектором. Содержит поля size – типа size_t, хранящее размер вектора, и infa – указатель на массив типа T, хранящий элементы вектора.

```
Moevector()
```

Конструктор класса Moevector. Не принимает параметров. Инициализирует size = 0 и infa = nullptr. Создает пустой вектор.

```
Moevector(size_t size)
```

Конструктор класса Moevector. Принимает размер вектора. Устанавливает this->size = size, выделяет память infa = new Tsize и инициализирует все элементы нулями.

```
Moevector(size_t size, T* infa_)
```

Конструктор класса Moevector. Принимает размер и указатель на массив. Устанавливает this->size = size, выделяет память infa = new T[size], затем в цикле for от 0 до size-1 копирует элементы infa_[i] в infa[i].

```
Moevector(const Moevector& other)
```

Конструктор копирования класса Moevector. Принимает ссылку на другой вектор. Копирует size = other.size. Если other.infa не равен nullptr, выделяет память и в цикле for копирует элементы. Если other.infa равен nullptr, устанавливает infa = nullptr.

```
~Moevector()
```

Деструктор класса Moevector. Не принимает параметров. Освобождает память.

```
friend ostream& operator<<(ostream& os, const Moevector& v)
```

Дружественная функция класса Moevector. Принимает поток вывода и константную ссылку на вектор. Выводит "Razmer: ", v.size, "\nVector: ", затем в цикле for от 0 до v.size-1 выводит v.infa[i] и пробел. Возвращает поток os.

```
Moevector& operator=(const Moevector& other)
```

Оператор присваивания класса Moevector. Принимает ссылку на другой вектор. Копирует size = other.size, удаляет старую память delete[] infa, выделяет новую infa = new T[size], в цикле for копирует элементы, возвращает *this.

```
Moevector& operator-=(const Moevector& lhs)
```

Оператор вычитания с присваиванием класса Moevector. Принимает ссылку на левый операнд. В цикле for от 0 до size-1 выполняет infa[i] -= lhs.infa[i]. Возвращает *this.

```
friend Moevector operator*(T mnozh, Moevector hh)
```

Дружественная функция умножения вектора на число. Принимает множитель mnozh и вектор hh. Создает новый вектор result размера hh.size. В цикле for от 0 до hh.size-1 выполняет result.infa[i] = hh.infa[i] * mnozh. Возвращает result.

```
T& operator[](size_t index)
```

Оператор доступа по индексу класса Moevector. Принимает индекс. Если $index \geq size$, выводит "Error index". Возвращает `infa[index]`.

```
void swap(Moevector<T>& other)
```

Метод класса Moevector. Принимает ссылку на другой вектор. Обменивает `size` и `infa` текущего вектора и переданного через временные переменные.

```
template <typename T> class Moematrix
```

Шаблонный класс, предназначенный для работы с квадратной матрицей. Содержит поля `msize` – типа `size_t`, хранящее размер матрицы, и `minfa` – типа `Moevector<Moevector<T>>`, хранящее строки матрицы.

```
Moematrix()
```

Конструктор класса Moematrix. Не принимает параметров. Инициализирует `msize = 0`.

```
Moematrix(size_t size)
```

Конструктор класса Moematrix. Принимает размер матрицы. Устанавливает `msize = size`, инициализирует `minfa = Moevector<Moevector<T>>(size)`, затем в цикле `for` от 0 до `size-1` выполняет `minfa[i] = Moevector<T>(size)`.

```
Moematrix(size_t size, Moevector<Moevector<T>>& infa)
```

Конструктор класса Moematrix. Принимает размер и ссылку на вектор векторов. Устанавливает `msize = size` и `minfa = infa`.

```
Moevector<T>& operator[](size_t index)
```

Оператор доступа класса Moematrix. Принимает индекс. Возвращает `minfa[index]`.

```
friend ostream& operator<<(ostream& os, const Moematrix<T>& m)
```

Дружественная функция вывода матрицы. Принимает поток и константную ссылку на матрицу. В цикле `for` от 0 до `m.msize-1` выводит `m.minfa[i]` и символ перевода строки. Возвращает поток `os`.

```
template <typename T> void bswap(T& a, T& b)
```

Шаблонная функция обмена значениями. Принимает ссылки на два элемента. Создает временную переменную `T temp = a`, затем `a = b`, затем `b = temp`.

```
template <typename T> class slau
```

Шаблонный класс, наследник `Moematrix<T>`, предназначен для решения систем линейных уравнений. Содержит только публичный конструктор и метод.

```
slau(Moevector<Moevector<T>> infa_, size_t size_)
```

Конструктор класса slau. Принимает матрицу и размер. Вызывает конструктор родителя `Moematrix<T>(size_, infa_)`.

```
Moevector<T> gauss(Moevector<T> b)
```

Метод класса slau. Принимает вектор правых частей `b`, возвращает вектор решений. В начале создает вектор `reshenie` размера `size`. В прямом ходе для `i` от 0 до `size-1` выполняет поиск `maxrow` с максимальным элементом в столбце `i`, при необходимости меняет строки через `swap` и `bswap`, проверяет диагональный элемент на ноль и выбрасывает исключение `runtime_error("Matritsa virozhdena")`, затем для `j` от `i+1` до `size-1` вычисляет `mnoz = (this)[j][i] / (this)[i][i]` и выполняет `(this)[j] -= mnoz * (this)[i]` и `b[j] -= mnoz * b[i]`. В обратном ходе для `i` от `size-1` до 0 вычисляет сумму `sum += (this)[i][j] * reshenie[j]` и `reshenie[i] = (b[i] - sum) / (this)[i][i]`. Возвращает `reshenie`.

```
int main()
```

Функция main. Объявляет `size_t size`, выводит "Vvedite razmer matritsi", вводит `size`. Создает `matrix` как `Moevector<Moevector<double>>(size)`. В цикле от 0 до `size-1` выполняет `matrix[i] = Moevector<double>(size)`. Создает `b` как `Moevector<double>(size)`. Выводит "Vvedite elementi matritsi A", в двух циклах вводит `matrix[i][j]`. Выводит "Vvedite vector b", в цикле вводит `b[i]`. В блоке `try` создает `slau<double> system(matrix, size)`, вызывает `system.gauss(b)`, сохраняет результат в `reshenie`, выводит "Reshenie sistemi: " и `reshenie`. В `catch(exception e)` выводит "oshibka: " и `e.what()`. Возвращает 0.

Результаты экспериментов

В ходе выполнения лабораторной работы был проведён ряд вычислительных экспериментов, позволяющих проверить корректность работы программы, исследовать особенности метода Гаусса с выбором ведущего элемента и проанализировать обработку вырожденных систем.

Эксперименты проводились для различных типов систем линейных уравнений:

- 1) Системы с невырожденной матрицей
- 2) Системы с нулевым диагональным элементом
- 3) Системы с вырожденной матрицей
- 4) Системы с полностью нулевой строкой
- 5) Системы с плохо обусловленной матрицей

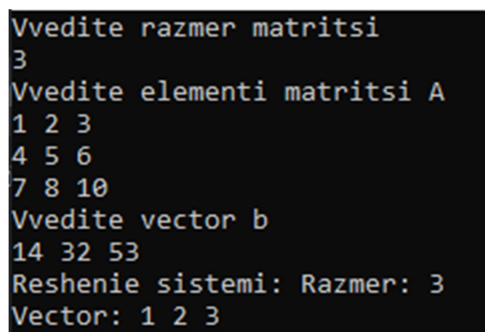
Особое внимание уделялось корректности перестановки строк при выборе ведущего элемента и обработке исключительных ситуаций с помощью исключений.

Система с невырожденной матрицей.

В программу была передана следующая система уравнений для проверки базового функционала

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 14 \\ 4x_1 + 5x_2 + 6x_3 = 32 \\ 7x_1 + 8x_2 + 10x_3 = 53 \end{cases}$$

В результате работы программы был получен ответ: $x_1 = 1; x_2 = 2; x_3 = 3$, что соответствует решению системы уравнений, значит программа справляется с невырожденными матрицами. На рис. 3 представлена консоль с вводом данных и ответом программы.



```
Vvedite razmer matritsi
3
Vvedite elementi matritsi A
1 2 3
4 5 6
7 8 10
Vvedite vector b
14 32 53
Reshenie sistemi: Razmer: 3
Vector: 1 2 3
```

Рис. 3 Результаты первого эксперимента

Система с нулевым диагональным элементом

В программу была передана следующая система уравнений:

$$\begin{cases} x_2 + x_3 = 3 \\ x_1 + x_2 + x_3 = 4 \\ x_1 + x_2 + 2x_3 = 6 \end{cases}$$

В результате работы программы был получен ответ $x_1 = 1; x_2 = 1; x_3 = 2$, что совпадает с решением системы. Это показывает, что в коде корректно реализован выбор ведущего элемента. При появлении нуля на диагонали программа переставляет строки матрицы. На рис. 4 сводка о работе программы.

```
Vvedite razmer matritsi
3
Vvedite elementi matritsi A
0 1 1
1 1 1
1 1 2
Vvedite vector b
3 4 6
Reshenie sistemi: Razmer: 3
Vector: 1 1 2
```

Рис. 4 Результаты второго эксперимента

Система с вырожденной матрицей

В программу была передана следующая система линейных уравнений:

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 1 \\ 2x_1 + 4x_2 + 6x_3 = 2 \\ 3x_1 + 6x_2 + 9x_3 = 3 \end{cases}$$

Особенность этой системы уравнений в том, что вторая и третья строки получены из первой умножением на 2 и 3 соответственно. Это означает, что определитель матрицы равен нулю, а значит матрица вырождена. Программа корректно распознала, что система вырождена и вывела соответствующее сообщение в консоль. На рис. 5 представлена сводка о третьем эксперименте.

```
Vvedite razmer matritsi
3
Vvedite elementi matritsi A
1 2 3
2 4 6
3 6 9
Vvedite vector b
1 2 3
oshibka: Matritsa virozhdena
```

Рис. 5 Результаты третьего эксперимента

Система с нулевой строкой

В программу передали следующую систему уравнений:

$$\begin{cases} 0 * x_1 + 0 * x_2 + 0 * x_3 = 1 \\ 2x_1 + 4x_2 + 6x_3 = 2 \\ 3x_1 + 6x_2 + 9x_3 = 3 \end{cases}$$

Система вырождена так как первая строка нулевая. Программа распознала, что матрица вырождена и вывела ошибку. На рис. 6 показана консоль.

```
Vvedite razmer matritsi
3
Vvedite elementi matritsi A
0 0 0
2 4 6
3 6 9
Vvedite vector b
1 2 3
oshibka: Matritsa virozhdena
```

Рис. 6 Результаты 4 эксперимента

Система с плохо обусловленной матрицей

В программу была передана следующая система уравнений

$$\begin{cases} 1.0000001x_1 + x_2 = 2.0000001 \\ x_1 + 0.9999999x_2 = 1.9999999 \end{cases}$$

В результате работы программы были получены ответы: $x_1 = 1.01124$; $x_2 = 0.988764$, что не соответствует решению системы, так как верное решение это $x_1 = 1$; $x_2 = 1$. Так вышло из-за того, что матрица системы является плохо обусловленной – её определитель близок к нулю. При вычитании близких чисел в процессе исключения неизвестных происходит потеря значащих цифр, что приводит к накоплению погрешности округления. Данный эксперимент наглядно демонстрирует ограниченность точности вычислений с плавающей точкой и чувствительность метода Гаусса к ошибкам округления для плохо обусловленных систем. На рис. 7 представлена консоль после вычислений.

```
Vvedite razmer matritsi
2
Vvedite elementi matritsi A
1.0000001 1
1 0.9999999
Vvedite vector b
2.0000001 1.9999999
Reshenie sistemi: Razmer: 2
Vector: 1.01124 0.988764
```

Рис. 7 Результаты пятого эксперимента

Заключение

Таким образом, в ходе выполнения лабораторной работы была реализована программа для решения систем линейных алгебраических уравнений методом Гаусса с выбором ведущего элемента. В процессе экспериментов было установлено, что для хорошо обусловленных и невырожденных матриц программа выдает результат с высокой точностью, соответствующий теоретическому решению. Однако на примере плохо обусловленной матрицы была наглядно продемонстрирована чувствительность метода Гаусса к погрешностям округления: вместо точного решения $(1, 1)$ были получены значения $(1.01124, 0.988764)$. Это объясняется ограниченной точностью представления чисел с плавающей точкой (double) и потерей значащих цифр при вычитании близких чисел. Для вырожденных матриц программа корректно выбрасывает исключение. Таким образом, реализованный алгоритм является работоспособным, но его точность существенно зависит от числа обусловленности матрицы.

Приложение

```
int main() {
    size_t size;
    cout << "Vvedite razmer matritsi" << '\n';
    cin >> size;
    Moevector<Moevector<double>> matrix(size);
    for (size_t i = 0; i < size; i++) {
        matrix[i] = Moevector<double>(size);
    }
    Moevector<double> b(size);
    cout << "Vvedite elementi matritsi A" << '\n';
    for (size_t i = 0; i < size; i++) {
        for (size_t j = 0; j < size; j++) {
            cin >> matrix[i][j];
        }
    }
    cout << "Vvedite vector b" << '\n';
    for (int i = 0; i < size; i++)
        cin >> b[i];
    try {
        slau<double> system(matrix, size);
        Moevector<double> reshenie = system.gauss(b);
        cout << "Reshenie sistemi: " << reshenie;
    }
    catch (exception e) { cout << "oshibka: " << e.what(); }
}
```